

Optimization of CLF–CBF–QP Control

Prajwal Dutta

Manufacturing Systems and Networks
Arizona State University
Mesa, AZ, USA
pdutta9@asu.edu

Wenlong Zhang

Manufacturing Systems and Networks
Arizona State University
Mesa, AZ, USA
Wenlong.Zhang@asu.edu

Abstract—We study safety and connectivity maintenance in decentralized multi-agent systems using a Control Lyapunov Function–Control Barrier Function–Quadratic Program (CLF–CBF–QP) formulation. The setting is a team of single-integrator robots moving in a cluttered planar workspace with line-of-sight (LOS) communication constraints. At every time step, a convex quadratic program modifies a nominal goal-seeking controller just enough to satisfy safety, collision-avoidance, and LOS connectivity constraints expressed as control barrier functions.

The main focus of this project is not the design of new CBFs, but the behavior of different numerical solvers on the resulting safety-critical QP. We compare MATLAB `quadprog` (interior-point), CVX (conic interior-point via DCP modeling), and OSQP (operator-splitting ADMM) on the same CLF–CBF–QP instances, and use the Karush–Kuhn–Tucker (KKT) conditions to interpret the constraints that are active at the optimum. Simulations with up to 70 robots show that all three solvers produce very similar safety and connectivity behavior, but with noticeably different runtime, numerical robustness, and trajectory deviations. In this implementation, `quadprog` emerges as a reliable baseline, while OSQP offers additional speed at the cost of more careful tuning.

Index Terms—Control Barrier Functions, Control Lyapunov Functions, Quadratic Programming, Multi-Agent Systems, Connectivity Maintenance, Optimization.

I. INTRODUCTION AND BACKGROUND

A. Broader Context and Motivation

The project sits inside a larger effort aimed at long-duration exploration of under-ice and subsurface environments using heterogeneous fleets of autonomous underwater vehicles (AUVs). In the MOTHERSHIP concept, a larger carrier platform supports and coordinates a collection of smaller AUVs to obtain dense measurements in regions such as ice-sheet grounding zones [1], [2]. In these environments, communication is acoustic, intermittent, and range-limited, and traditional infrastructure such as GPS is not available [3]–[5].

The author’s PhD work focuses on connectivity-aware coordination for such fleets in dynamic ocean flows, including how AUVs can exploit environmental transport for energy-efficient navigation and rendezvous [6]–[8]. Classical transport models such as advection–diffusion and turbulent dispersion [9]–[11] and recent results on Lagrangian coherent structures for marine robots [7], [8] provide the physical backdrop.

The course project reported here takes a smaller, more abstract slice of that problem: we look at a planar multi-robot system with LOS-limited communication, and ask how to

maintain safe, connected motion while robots move to goals in the presence of obstacles. The intent is that the tools developed here—CLF–CBF–QP control and solver analysis—can later be carried over to more realistic underwater models.

B. Decentralized Coordination and Connectivity

Decentralized coordination of multi-agent systems is often modeled on interaction graphs, with edges representing communication or sensing links. Consensus and formation-control results rely heavily on the connectivity of the graph, typically through properties of the Laplacian matrix [12]. If connectivity is lost, guarantees on convergence and information flow can fail.

A number of decentralized controllers have been proposed to maintain connectivity while the agents move. Some directly regulate the algebraic connectivity (the second smallest Laplacian eigenvalue) [13], [14]; others rely on potential fields or gradient-based rules that penalize long edges or topological changes [15]–[17]. In swarm and underwater robotics, these controllers must be combined with realistic communication and platform models: for example, implicit 3D coordination in fish-inspired underwater swarms [18], role-based addressing for AUV fleets [4], and hybrid mothership–passenger architectures [2].

C. Safety-Critical Control via CLF–CBF–QP

Control Lyapunov Functions (CLFs) and Control Barrier Functions (CBFs) give a clean way to encode goal convergence and safety as inequalities that are affine in the control input. Ames *et al.* showed that combining these ingredients in a QP leads to a practical safety filter: at each step, the QP computes the control input closest to a nominal one that still satisfies CLF and CBF conditions [19], [20]. Xu *et al.* studied how such controllers behave under model errors and noise [21], and Mestres *et al.* used CLF–CBF–QP formulations to minimally modify motion plans when safety constraints become tight [22].

In that view, the CLF encodes “what we want to do” (reach a goal), while the CBFs encode “what we must not violate” (collisions, workspace boundaries, connectivity). The QP resolves conflicts between them by introducing slack variables and weights in the cost. In this project, we adopt this CLF–CBF–QP structure and ask: given a fixed formulation,

how do different QP solvers behave when they are used as the inner loop of a safety-critical controller?

II. LITERATURE REVIEW

This section briefly positions the project relative to prior work on CLF–CBF control, underwater multi-robot systems, and QP solvers for control.

A. Optimization and CLF–CBF-Based Multi-Robot Control

The original CLF–CBF–QP safety filter of Ames *et al.* [19], [20] formulates safety constraints as CBF inequalities and allows CLF constraints to be relaxed through slack variables. The resulting QP is convex and can be solved at high rates on embedded hardware. Xu *et al.* [21] examined robustness of CBF-based QP controllers and provided conditions under which safety is preserved despite uncertainty. Mestres *et al.* [22] showed how CLF–CBF–QP can serve as a “safety shield” around nominal planners.

For connectivity, Sabattini *et al.* [15] and Capelli *et al.* [16], [17] constructed CBFs that keep graph connectivity above a threshold while robots move. Their formulations are conceptually close to the range-based and LOS-based CBFs used here, although they do not focus on solver-level behavior or larger team sizes.

B. Multi-AUV and Underwater Robotics Context

Underwater multi-robot systems bring additional constraints from acoustic communication, limited bandwidth, and complex flow fields. Quattrini Li *et al.* [3] review underwater communication technologies and emphasize the need for robust, decentralized strategies. Schiel and collaborators proposed role-based network addressing and orthogonal chirp-based communication schemes tailored to AUV fleets [4], [5]. Swarm-style and mothership–passenger coordination have been demonstrated experimentally in underwater settings [1], [2], [18]. The author’s prior work [6] combines environmental transport models [9]–[11] with connectivity-aware control, which motivated the simplified LOS connectivity model used in this paper.

While these studies rely on optimization and graph-theoretic tools, they typically do not combine CLF–CBF–QP with an explicit comparison of QP solvers.

C. QP Solvers and Numerical Optimization

From a numerical standpoint, CLF–CBF controllers lead to convex QPs that must be solved repeatedly, with slowly varying data and many inequality constraints. Classical references such as Nocedal and Wright [23] survey interior-point and active-set methods and discuss how they solve the KKT system for QPs.

Several solvers have been tailored to control applications. qpOASES is an active-set solver tuned for model predictive control (MPC) and parametric QPs, exploiting warm-start and homotopy ideas [24]. HPIPM is a high-performance interior-point framework specifically designed for optimal control problems [25]. OSQP takes a different route: it uses

ADMM on a reformulated KKT system and leverages cached factorization and simple projections for speed [26]. On the modeling side, disciplined convex programming [27] underlies CVX [28], which lets users specify convex problems in a high-level form and then translates them to a cone program for an interior-point solver such as SDPT3.

In robotics, general-purpose solvers such as MATLAB `quadprog` remain widely used for CLF–CBF QPs, while OSQP, qpOASES, and HPIPM are increasingly adopted in embedded MPC implementations [24]–[26]. Existing comparisons mostly focus on standard MPC benchmarks; systematic tests on safety-critical CLF–CBF–QP problems with dense CBF constraints and LOS connectivity are less common. This project contributes an empirical comparison of `quadprog`, CVX, and OSQP on such a problem.

III. PROBLEM STATEMENT

We consider N robots with single-integrator dynamics in a planar environment. Robot i has position $x_i \in \mathbb{R}^2$, control input $u_i \in \mathbb{R}^2$, and dynamics

$$\dot{x}_i = u_i, \quad \|u_i\|_\infty \leq u_{\max}. \quad (1)$$

Each robot is assigned a goal $g_i \in \mathbb{R}^2$ and moves in a 10×10 m workspace populated with static rectangular obstacles.

The controller must satisfy four requirements at all times:

- 1) *Task completion*: each robot should converge to its goal.
- 2) *Safety*: robots must avoid collisions with other robots and with obstacles.
- 3) *Connectivity*: LOS-based communication should be preserved through a sparse edge set derived from the LOS graph.
- 4) *Feasibility*: control inputs must remain within actuator bounds.

Connectivity is modeled by an LOS graph $G_{\text{LOS}}(x)$ whose vertices are robots and whose edges connect robots that are within range and not occluded by obstacles. At each step, a Minimum Connectivity Spanning Tree) is extracted from $G_{\text{LOS}}(x)$ using edge weights that depend on distance and nominal control, following classical MST ideas [29]–[36]. Connectivity CBFs are enforced only on these tree edges.

Safety, obstacle avoidance, and connectivity constraints are encoded using CBFs, while goal convergence is encoded through a CLF, following [16], [17], [19]–[22]. At each time step, we solve a convex QP whose decision variables are:

- $u \in \mathbb{R}^{2N}$: stacked robot controls,
- $\delta \in \mathbb{R}_{\geq 0}^{m_{\text{cbf}}}$: CBF slacks,
- $\varepsilon \in \mathbb{R}_{\geq 0}^N$: CLF slacks.

The QP cost penalizes deviation from a nominal goal-tracking controller and the use of slack variables, subject to linear CBF and CLF inequalities and box constraints on u .

The rest of the paper focuses on this QP: its structure, the associated KKT conditions, and the numerical behavior of three solvers (`quadprog`, CVX, OSQP) when the QP is solved at every control step.

IV. SYSTEM MODEL AND CLF-CBF-QP

We stack all robot states and controls as

$$x = [x_1^\top, \dots, x_N^\top]^\top, \quad u = [u_1^\top, \dots, u_N^\top]^\top \in \mathbb{R}^{2N}. \quad (2)$$

For any pair (i, j) we denote the distance and unit vector

$$d_{ij} = \|x_i - x_j\|_2, \quad n_{ij} = \frac{x_i - x_j}{d_{ij}} \quad (d_{ij} > 0), \quad (3)$$

and for a robot-obstacle pair (i, o) with obstacle center c_o we set

$$d_{io} = \|x_i - c_o\|_2, \quad n_{io} = \frac{x_i - c_o}{d_{io}}. \quad (4)$$

The QP decision vector at each time step is

$$z = \begin{bmatrix} u \\ \delta \\ \varepsilon \end{bmatrix}, \quad u \in \mathbb{R}^{2N}, \quad \delta \in \mathbb{R}_{\geq 0}^{m_{\text{cbf}}}, \quad \varepsilon \in \mathbb{R}_{\geq 0}^N. \quad (5)$$

Each robot has a goal $g_i \in \mathbb{R}^2$ and a nominal proportional controller

$$u_{i,\text{nom}} = -k_p(x_i - g_i), \quad k_p > 0, \quad (6)$$

clipped to satisfy $\|u_i\|_\infty \leq u_{\max}$. The aggregate CLF

$$V(x) = \sum_{i=1}^N \|x_i - g_i\|^2 \quad (7)$$

has Lie derivative

$$\dot{V}(x, u) = \sum_{i=1}^N 2(x_i - g_i)^\top u_i. \quad (8)$$

Per-robot CLF inequalities are written as

$$2(x_i - g_i)^\top u_i \leq -\gamma_{\text{clf}} \|x_i - g_i\|^2 + \varepsilon_i, \quad (9)$$

with $\varepsilon_i \geq 0$.

The QP objective is

$$\min_{u, \delta, \varepsilon} \|u - u_{\text{nom}}\|^2 + p_{\text{cbf}} \|\delta\|^2 + p_{\text{clf}} \|\varepsilon\|^2, \quad (10)$$

where p_{cbf} and p_{clf} weight violations of CBF and CLF inequalities more heavily than deviation from the nominal control. In quadratic form,

$$J(z) = \frac{1}{2} z^\top H z + f^\top z, \quad (11)$$

with

$$H = \text{diag}(2I_{2N}, 2p_{\text{cbf}}I_{m_{\text{cbf}}}, 2p_{\text{clf}}I_N), \quad (12)$$

$$f = \begin{bmatrix} -2u_{\text{nom}} \\ 0 \\ 0 \end{bmatrix}. \quad (13)$$

For any smooth barrier function $h(x) \geq 0$ we use the standard CBF condition

$$\nabla h(x)^\top u \geq -\alpha h(x), \quad (14)$$

which is affine in u under the single-integrator model [19], [20]. In this project we employ:

- *Inter-robot safety* (keep $d_{ij} \geq R_s$):

$$h_{ij}^{\text{safe}} = d_{ij} - R_s, \quad n_{ij}^\top (u_i - u_j) \geq -\alpha_s (d_{ij} - R_s). \quad (15)$$

- *Connectivity* (LOS neighbors within R_c):

$$h_{ij}^{\text{conn}} = R_c - d_{ij}, \quad n_{ij}^\top (u_i - u_j) \leq \alpha_c (R_c - d_{ij}), \quad (16)$$

- *Obstacle avoidance* (stay outside R_{obs}):

$$h_{io}^{\text{obs}} = d_{io} - R_{\text{obs}}, \quad n_{io}^\top u_i \geq -\alpha_o (d_{io} - R_{\text{obs}}). \quad (17)$$

Stacking all CBF and CLF inequalities yields

$$A_{\text{cbf}}(x) u - \delta \leq b_{\text{cbf}}(x), \quad \delta \geq 0, \quad (18)$$

$$A_{\text{clf}}(x) u - \varepsilon \leq b_{\text{clf}}(x), \quad \varepsilon \geq 0, \quad (19)$$

and the box constraints on u are

$$-u_{\max} \mathbf{1} \leq u \leq u_{\max} \mathbf{1}. \quad (20)$$

Combining (10) with these linear inequalities gives the compact CLF-CBF-QP used throughout:

$$\begin{aligned} \min_{u, \delta, \varepsilon} \quad & \|u - u_{\text{nom}}\|^2 + p_{\text{cbf}} \|\delta\|^2 + p_{\text{clf}} \|\varepsilon\|^2 \\ \text{s.t.} \quad & A_{\text{cbf}}(x) u - \delta \leq b_{\text{cbf}}(x), \\ & A_{\text{clf}}(x) u - \varepsilon \leq b_{\text{clf}}(x), \\ & -u_{\max} \mathbf{1} \leq u \leq u_{\max} \mathbf{1}, \\ & \delta \geq 0, \quad \varepsilon \geq 0. \end{aligned} \quad (21)$$

V. KKT CONDITIONS AND INTERPRETATION

For a convex QP of the form

$$\begin{aligned} \min_x \quad & \frac{1}{2} x^\top H x + f^\top x \\ \text{s.t.} \quad & A x \leq b, \quad lb \leq x \leq ub, \end{aligned} \quad (22)$$

the Karush-Kuhn-Tucker (KKT) conditions characterize optimal solutions. Let $\lambda \geq 0$ be Lagrange multipliers for $Ax \leq b$, and $v_\ell, v_u \geq 0$ multipliers for the lower and upper bounds. The Lagrangian is

$$L = \frac{1}{2} x^\top H x + f^\top x + \lambda^\top (Ax - b) + v_\ell^\top (lb - x) + v_u^\top (x - ub). \quad (23)$$

The KKT conditions are:

$$\text{Stationarity:} \quad Hx + f + A^\top \lambda - v_\ell + v_u = 0, \quad (24)$$

$$\text{Primal feasibility:} \quad Ax \leq b, \quad lb \leq x \leq ub, \quad (25)$$

$$\text{Dual feasibility:} \quad \lambda, v_\ell, v_u \geq 0, \quad (26)$$

$$\text{Complementary slackness:} \quad \lambda_i (A_i x - b_i) = 0, \quad (27)$$

$$v_{\ell, i} (lb_i - x_i) = 0, \quad (28)$$

$$v_{u, i} (x_i - ub_i) = 0. \quad (29)$$

In the CLF-CBF-QP, these conditions have a straightforward physical meaning. A positive multiplier on a safety CBF row means that two robots are exactly on the safety boundary and that constraint is actively steering their relative motion. Positive multipliers on connectivity rows indicate edges that are about to break and are being “held together” by the

controller. Nonzero bound multipliers correspond to saturated control inputs at $\pm u_{\max}$. Slack variables δ, ε become nonzero only when constraints are in conflict, and their magnitudes measure how much we are willing to relax safety or convergence to keep the QP feasible.

Because (21) is convex with $H \succeq 0$ and linear constraints, KKT conditions are both necessary and sufficient: any point that satisfies them is the unique global minimizer.

VI. QP SOLVERS

The QP (21) has diagonal H , a sparse constraint matrix $A = [A_{\text{cbf}} \ A_{\text{clf}}]$, and simple bounds on u, δ , and ε . All three solvers considered here try to satisfy the KKT conditions, but they do so using different numerical strategies.

A. MATLAB quadprog (Interior-Point)

With `Algorithm='interior-point-convex'`, `quadprog` uses a primal-dual interior-point method [23]. Inequality constraints $Ax \leq b$ are handled through a logarithmic barrier, leading to a barrier-regularized problem

$$\min_x \frac{1}{2}x^\top Hx + f^\top x - \mu \sum_i \log(b_i - A_i x), \quad (30)$$

with $\mu > 0$ the barrier parameter. The solution of this problem for a fixed μ lies on the so-called central path; as μ is reduced, the solutions approach the true KKT point.

Each outer iteration forms and factorizes a sparse KKT system involving H, A , and current slack variables, then performs a Newton step and line search to keep $b_i - A_i x > 0$. Bounding constraints are enforced explicitly. In our case, the KKT matrix benefits from the diagonal H and the sparsity pattern of A , since each CBF row couples at most two robots. Warm-starting from the previous solution further reduces the number of iterations. Over the runs considered, `quadprog` delivered KKT-consistent solutions without numerical breakdowns.

B. CVX (Disciplined Convex Programming)

CVX [27], [28] is a modeling layer rather than a solver. The user specifies (21) in a high-level syntax, and CVX checks convexity using disciplined convex programming rules before converting the problem into a cone program. A generic interior-point solver such as SDPT3 is then called to solve the cone formulation.

In this project, CVX is used as a reference: it provides a high-accuracy solution that is convenient for checking that the CLF-CBF-QP has been implemented correctly, and for validating the objective value and slack patterns obtained from the other solvers. Its runtime is substantially higher than that of `quadprog` or OSQP, so it is not a candidate for real-time control.

C. OSQP (Operator-Splitting ADMM)

OSQP [26] solves QPs by applying ADMM to a split KKT system. After rewriting (21) in the canonical form

$$\min_x \frac{1}{2}x^\top Px + q^\top x \quad \text{s.t.} \quad \ell \leq Ax \leq u, \quad (31)$$

OSQP carries out the iterations

$$x^{k+1} = (P + \rho I)^{-1}(A^\top(z^k - y^k) - q), \quad (32)$$

$$z^{k+1} = \Pi_{[\ell, u]}(Ax^{k+1} + y^k), \quad (33)$$

$$y^{k+1} = y^k + Ax^{k+1} - z^{k+1}, \quad (34)$$

where $\Pi_{[\ell, u]}$ is projection onto the constraint box and $\rho > 0$ is a penalty parameter. The factorization of $(P + \rho I)$ is cached and reused, which is effective when the QP structure is fixed and only the right-hand side changes from one time step to the next.

For the CLF-CBF-QP, $P = H$ is diagonal and A does not change with time, so OSQP can exploit exactly this reuse. In our experiments, it typically converged in a modest number of iterations and achieved the fastest average solve time. However, because ADMM is a first-order method with asymptotic convergence, the accuracy depends strongly on the chosen tolerances and ρ . We observed occasional iterations where the stopping criteria were met but the KKT residuals remained larger than those from `quadprog` or CVX; in such cases OSQP's objective value and trajectories deviated more noticeably.

D. Summary of Solver Roles

In summary, `quadprog` provides a robust and interpretable baseline with direct access to KKT multipliers, CVX serves as a high-confidence reference for debugging and validation, and OSQP illustrates the potential and challenges of using operator-splitting methods for fast CLF-CBF-QP control.

VII. SIMULATION SETUP AND RESULTS

A. Scenario and Experimental Design

We simulate $N = 70$ robots in a 10×10 m workspace with five axis-aligned rectangular obstacles. Robots are initialized in four clusters and assigned fixed goal locations. The safety radius is $R_s = 0.25$ m, the communication range is $R_c = 4.0$ m, and the obstacle radius is $R_{\text{obs}} = 0.8$ m. The control bound is $u_{\max} = 3.0$ m/s. LOS connectivity is enforced on line of sight edges built from the LOS graph at each step.

At every control step we solve the same CLF-CBF-QP in (21) using three solvers:

- MATLAB `quadprog` (interior-point-convex),
- CVX (cone interior-point, SDPT3 backend),
- OSQP (ADMM).

All runs use identical initial conditions, goals, and CLF/CBF parameters. The robots converge to their goals by iteration 79; we keep running the solvers on the same horizon for comparison.

Figure 1 illustrates the initial condition for the two scenarios considered: robots start in four dense clusters and must move toward their assigned goals either in a static workspace or in the presence of a vortex-like flow field. The corresponding trajectories and final configurations are shown in Fig. 2, where the dashed lines trace the Minimum Spanning Tree edges that are enforced by the CBF constraints.

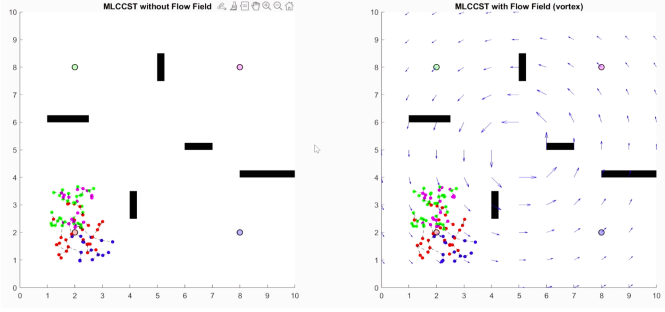


Fig. 1. Initial configurations for the two scenarios. Left: Robot fleet without flow dynamics; robots start in four tight clusters and communicate over LOS edges only. Right: Robot fleet with a vortex flow field superimposed, showing how the environmental drift interacts with the initial communication graph. Obstacles are shown in black and goal locations as hollow circles.

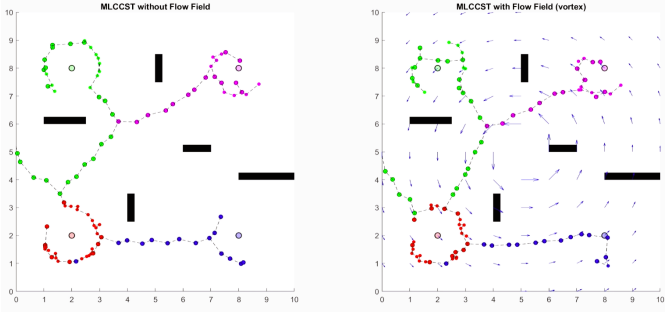


Fig. 2. Representative trajectories and final positions under the CLF-CBF-QP controller. Left: Robot fleet without flow; robots spread from their clusters, navigate around obstacles, and reach their goals while maintaining LOS connectivity. Right: Robot fleet with vortex flow; trajectories are distorted by the background flow, but the optimization still preserves connectivity and brings all robots to their goals.

B. Metrics

For each solver we record, at every iteration:

- the solve time in milliseconds;
- the QP objective value

$$J = \|u\|^2 + p_{\text{cbf}}\|\delta\|^2 + p_{\text{clf}}\|\varepsilon\|^2;$$

- the maximum CBF slack $\max_i \delta_i$ (largest safety/connectivity relaxation);
- the deviation of robot trajectories from the `quadprog` solution, measured as the Frobenius norm between stacked positions.

C. Aggregate Numerical Comparison

Table I summarizes the main metrics over the full run.

To make timing dispersion and total computation more explicit, Table II reports standard deviation and cumulative solve time.

The data in Tables I and II lead to the following observations:

- **Runtime.** OSQP has the lowest average solve time (≈ 8.6 ms), followed by `quadprog` (≈ 30.1 ms), while CVX is roughly one to two orders of magnitude slower. Over the full horizon, OSQP accumulates ≈ 0.68 s of

TABLE I
SOLVER COMPARISON FOR $N = 70$ ROBOTS (SAME QPS, SAME SEED).

Solver	Avg time (ms)	Max time (ms)	Avg obj.	Pos. diff (m)
<code>quadprog</code> (IPM)	30.13	1101.20	5214.26	0.00
CVX (SDPT3)	784.90	7447.64	5111.55	0.78
OSQP (ADMM)	8.63	27.72	5112.53	0.64

All three solvers yield essentially the same maximum CBF slack $\max_i \delta_i \approx 1.7114$, indicating agreement on when safety and connectivity constraints are tightest.

TABLE II
TIMING DISPERSION AND CUMULATIVE COMPUTATION TIME.

Solver	Std time (ms)	Total time (ms)	Max CBF slack
<code>quadprog</code> (IPM)	123.93	2380.43	1.71135
CVX (SDPT3)	766.39	62006.98	1.71134
OSQP (ADMM)	7.09	681.98	1.71137

computation, `quadprog` ≈ 2.38 s, and CVX about 62 s. Relative to CVX, `quadprog` is about $26\times$ faster; OSQP is about $3.5\times$ faster than `quadprog`.

- **Solution quality.** Average objectives for CVX and OSQP are around 5.11×10^3 , slightly below but very close to the `quadprog` average of 5.21×10^3 . The time profiles of J (not shown here) track each other closely, which is consistent with all three solvers converging to essentially the same KKT point.
- **Slack usage.** The maximum CBF slack values and their peaks in time are nearly identical across solvers, reaching about 1.71 during the most constrained maneuvers. This suggests that the QP formulations and constraint activations agree and that differences between solvers do not cause qualitatively different safety behavior.
- **Trajectory deviation.** The average position deviation of CVX and OSQP from the `quadprog` baseline is on the order of 0.7 m in a 10 m workspace. These differences are visible but modest, and largely driven by different numerical tolerances rather than structural differences in the optimization problem.

D. Plots and Qualitative Interpretation

Figure 3 gathers several plots: solve-time distributions, cumulative computation time, trajectory deviation from `quadprog`, speedup factors, and objective traces. These visualizations reinforce the numerical trends:

- OSQP's solve-time distribution is tight and low; `quadprog` is somewhat slower with occasional spikes; CVX has wide, high-variance timings.
- Cumulative computation grows almost linearly, with slopes matching the average times in Table I.
- Objective traces and CBF slack peaks align well between solvers, suggesting that minor trajectory differences are due to numerical tolerance rather than qualitatively different control decisions.

Overall, the experiments show that `quadprog` offers a good compromise between robustness and speed, OSQP can

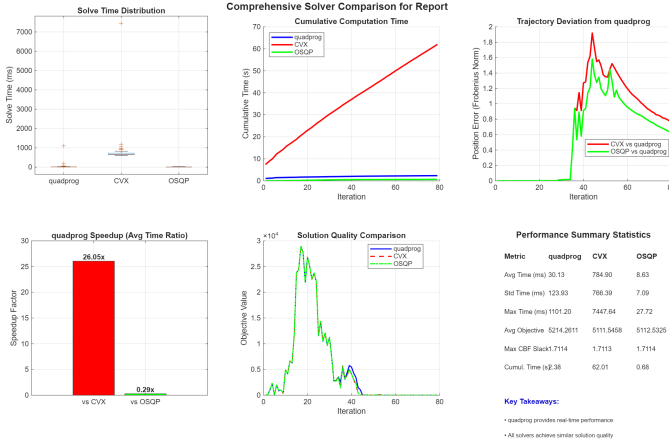


Fig. 3. Comprehensive solver comparison: (top) solve-time distribution, cumulative computation time, and trajectory deviation from the quadprog baseline; (bottom) quadprog speedup factors, objective traces, and summary statistics.

further reduce runtime with careful tuning, and CVX is best viewed as a reference tool rather than a real-time solver for this problem.

VIII. DISCUSSION AND FUTURE WORK

The CLF-CBF-QP formulation used here can be interpreted as a least-change projection of a nominal goal-seeking control input onto the set of control actions that respect safety, connectivity, obstacle, and input constraints. CBF inequalities act as directional speed limits along geometric normals, while CLF inequalities encourage progress to goals and can be relaxed when safety and connectivity must take priority. The KKT multipliers obtained from the QP give a compact summary of which constraints are steering the system at each time.

On the numerical side, the study highlights that solver choice matters, even when all solvers are applied to the same convex QP. Interior-point methods (quadprog, CVX) provide highly accurate primal-dual solutions that satisfy KKT conditions to tight tolerances, but with different overheads due to modeling layers and cone formulations. OSQP, based on ADMM, offers much lower average solve times and attractive features for embedded use, but requires more attention to parameter tuning and stopping criteria to avoid occasional numerical issues.

Several directions are natural next steps. One is to use the KKT multipliers more systematically in the analysis, for example to study how the set of active CBF constraints evolves over time and how often safety and connectivity are competing. Another is to tune OSQP specifically for this CLF-CBF-QP and explore polishing and scaling strategies, with the aim of safely replacing quadprog in the loop. Finally, extending the experiments to larger teams and more realistic AUV models, including communication delays and environmental flows, would bring the study closer to the full MOTHERSHIP setting.

REFERENCES

- [1] A. Muto, S. Athar, N. Butler, and Others, “Meshed observations of the remote subsurface with heterogeneous intelligent platforms (mothership),” in *AGU Fall Meeting Abstracts*, vol. 2024, 2024, pp. C33D-0466.
- [2] N. L. Butler and G. A. Hollinger, “Hybrid decentralization for multi-robot orienteering with mothership-passenger systems,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 6573–6579.
- [3] A. Quattrini Li, C. J. Carver, Q. Shao, X. Zhou, and S. Nelakuditi, “Communication for underwater robots: Recent trends,” *Current Robotics Reports*, vol. 4, no. 2, pp. 13–22, 2023.
- [4] B. Schiel and P. Lundrigan, “Role-based network addressing for fleets of autonomous underwater vehicles,” in *Proceedings of the 18th International Conference on Underwater Networks & Systems*, 2024.
- [5] B. Schiel and Others, “Using orthogonal chirps underwater for in-band, full-duplex communication,” in *Proceedings of the 18th International Conference on Underwater Networks & Systems*, 2024.
- [6] P. Dutta, “Connectivity-aware coordination and control of autonomous underwater vehicles in environmental dynamics,” Master’s thesis, Arizona State University, 2025.
- [7] C. Wei, H. G. Tanner, X. Yu, and M. A. Hsieh, “Low-range interaction periodic rendezvous along lagrangian coherent structures,” in *2019 American Control Conference (ACC)*, 2019, pp. 4012–4017.
- [8] G. Knizhnik, P. Li, X. Yu, and M. A. Hsieh, “Flow-based control of marine robots in gyre-like environments,” in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3047–3053.
- [9] G. I. Taylor, “Diffusion by continuous movements,” *Proceedings of the London Mathematical Society*, vol. 2, no. 1, pp. 196–212, 1922.
- [10] A. Okubo, “Oceanic diffusion diagrams,” *Deep Sea Research and Oceanographic Abstracts*, vol. 18, no. 8, pp. 789–802, 1971.
- [11] A. J. Majda and P. R. Kramer, “Simplified models for turbulent diffusion: Theory, numerical modelling, and physical phenomena,” *Physics Reports*, vol. 314, no. 4-5, pp. 237–574, 1999.
- [12] R. Olfati-Saber, J. A. Fax, and R. M. Murray, “Consensus and cooperation in networked multi-agent systems,” *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007.
- [13] M. C. De Gennaro and A. Jadbabaie, “Decentralized control of connectivity for multi-agent systems,” *IEEE Transactions on Automatic Control*, vol. 68, no. 6, pp. 1325–1339, 2023.
- [14] —, “Decentralized control of connectivity for multi-agent systems,” *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 45–60, 2023.
- [15] L. Sabattini, N. Chopra, and C. Secchi, “Decentralized connectivity maintenance for cooperative control of mobile robotic systems,” *The International Journal of Robotics Research*, vol. 32, no. 12, pp. 1411–1423, 2013.
- [16] B. Capelli and L. Sabattini, “Connectivity maintenance: Global and optimized approach through control barrier functions,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 5590–5596.
- [17] B. Capelli, H. Fouad, G. Beltrame, and L. Sabattini, “Decentralized connectivity maintenance with time delays using control barrier functions,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 1586–1592.
- [18] F. Berlinger, M. Gauci, and R. Nagpal, “Implicit coordination for 3d underwater collective behaviors in a fish-inspired robot swarm,” *Science Robotics*, vol. 6, no. 50, p. eabd8668, 2021.
- [19] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function-based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [20] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *2019 18th European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [21] X. Xu, P. Tabuada, J. Grizzle, and A. Ames, “Robustness of control barrier functions for safety-critical control,” in *8th IFAC Conference on Analysis and Design of Hybrid Systems*, 2015, pp. 54–61.
- [22] A. Mestres et al., “Safe motion planning via clf-cbf quadratic programs,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.
- [23] J. Nocedal and S. Wright, *Numerical Optimization*. Springer, 2006.
- [24] H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, “qpOases: A parametric active-set algorithm for quadratic programming,” *Mathematical Programming Computation*, vol. 6, no. 4, pp. 327–363, 2014.

- [25] G. Frison and M. Diehl, “Hpipm: A high-performance quadratic programming framework for model predictive control,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6563–6569, 2020.
- [26] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “Osqp: An operator splitting solver for quadratic programming,” *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.
- [27] M. Grant and S. Boyd, “Disciplined convex programming,” in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. Springer, 2006, pp. 155–210.
- [28] M. Grant, S. Boyd, and Y. Ye, “Cvx: Matlab software for disciplined convex programming,” <http://cvxr.com/cvx>, 2014, version 2.1.
- [29] O. Borůvka, “O jistém problému minimálním,” *Práce Moravské Přírodovědecké Společnosti*, 1926.
- [30] J. B. Kruskal, “On the shortest spanning subtree of a graph and the traveling salesman problem,” *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.
- [31] R. C. Prim, “Shortest connection networks and some generalizations,” *The Bell System Technical Journal*, vol. 36, no. 6, pp. 1389–1401, 1957.
- [32] R. G. Gallager, P. A. Humblet, and P. M. Spira, “A distributed algorithm for minimum-weight spanning trees,” *ACM Transactions on Programming Languages and Systems*, vol. 5, no. 1, pp. 66–77, 1983.
- [33] B. Awerbuch, “Optimal distributed algorithms for minimum weight spanning tree, counting, leader election, and related problems,” in *19th Annual ACM Symposium on Theory of Computing*, 1987, pp. 230–240.
- [34] M. Khan and G. Pandurangan, “A fast distributed approximation algorithm for minimum spanning trees,” *Distributed Computing*, vol. 20, no. 6, pp. 391–402, 2008.
- [35] D. B. Venkateswaran, Z. Qu, and A. Gusrialdi, “A distributed method for detecting critical edges and increasing edge connectivity in undirected networks,” in *2024 IEEE Conference on Decision and Control (CDC)*, 2024, pp. 6951–6956.
- [36] Y. Yang, Y. Lyu, and W. Luo, “Minimally constrained multi-robot coordination with line-of-sight connectivity maintenance,” *arXiv preprint arXiv:2303.04271*, 2023.